

⇐ CẨM NANG DATA ANALYST ⇒ TRANG 1

1. KIẾN THỨC NỀN TẢNG DATA ANALYTICS

★ Data Analytics là gì?

Data Analytics là quá trình thu thập, làm sạch, khám phá và phân tích dữ liệu nhằm tìm ra những quy luật hữu ích, rút ra kết luận và hỗ trợ đưa ra các quyết định kinh doanh tốt hơn.

★ Data Analyst - Vai trò

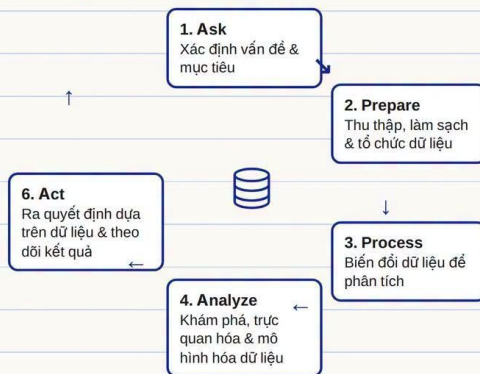
- Thu thập dữ liệu từ nhiều nguồn khác nhau
- Làm sạch và chuẩn bị dữ liệu
- Khám phá dữ liệu để tìm quy luật
- Tạo báo cáo & dashboard
- Đưa ra insight & đề xuất
- Hỗ trợ ra quyết định dựa trên dữ liệu



★ Các loại Analytics



★ Vòng đời Data Analytics



★ Kỹ năng cần thiết

- ✓ Analytical Thinking
- ✓ Problem Solving
- ✓ SQL Querying
- ✓ Excel Proficiency
- ✓ Data Visualization
- ✓ Statistics Basics
- ✓ Communication
- ✓ Business Understanding

★ Công cụ cho Data Analyst

- Excel
- SQL
- Python
- Power BI
- Tableau
- Google Sheets
- Jupyter Notebook

★ Quy trình làm việc điển hình của Data Analyst



★ Điểm nhanh phòng vấn

- Data Analyst biến dữ liệu thô thành những insight có ý nghĩa.

CẨM NANG DATA ANALYST

TRANG 2

2. KIẾN THỨC NỀN TẢNG EXCEL

Excel là công cụ mạnh mẽ để tổ chức, phân tích và trực quan hóa dữ liệu.
Kỹ năng Excel vững vàng là điều bắt buộc với mọi Data Analyst.

★ Thao tác cơ bản

- Nhập dữ liệu vào ô
- Dùng công thức để thực hiện tính toán
- Định dạng dữ liệu để dễ đọc
- Sắp xếp và lọc dữ liệu
- Dùng hàm để phân tích
- Tạo Table và Chart

★ Công thức & Hàm thường dùng

Hàm	Cú pháp	Ví dụ	Chức năng
SUM	=SUM(range)	=SUM(B2:B10)	Cộng tất cả các số trong một range
AVERAGE	=AVERAGE(range)	=AVERAGE(C2:C10)	Trả về giá trị trung bình của các số
COUNT	=COUNT(range)	=COUNT(A2:A10)	Đếm số ô chứa số
COUNTA	=COUNTA(range)	=COUNTA(A2:A10)	Đếm số ô không rỗng
IF	=IF(logical_test, value_if_true, value_if_false)	=IF(D2>=60, "Pass", "Fail")	Trả về giá trị dựa trên điều kiện
SUMIFS	=SUMIFS(sum_range, criteria_range1, criteria1, ...)	=SUMIFS(D2:D10, A2:A10, "East")	Tính tổng dựa trên một hoặc nhiều điều kiện
COUNTIFS	=COUNTIFS(range1, criteria1, ...)	=COUNTIFS(A2:A10, "East", C2:C10, ">=60")	Đếm dựa trên một hoặc nhiều điều kiện
XLLOOKUP	=XLLOOKUP(lookup_value, lookup_array, return_array, [if_not_found])	=XLLOOKUP(E2, A2:A10, B2:B10, "Not Found")	Tìm một giá trị trong range và trả về kết quả khớp
INDEX + MATCH	=INDEX(return_range, MATCH(lookup_value, lookup_range, 0))	=INDEX(B2:B10, MATCH(E2, A2:A10, 0))	Công cụ lookup mạnh mẽ (hoạt động theo mọi hướng)

★ Toán tử số học cơ bản

+	Cộng
-	Trừ
*	Nhân
/	Chia
^	Lũy thừa (Power)
%	Phần trăm

★ Các hàm quan trọng khác

- CONCAT / TEXTJOIN - Nối text từ nhiều ô.
- TRIM - Xóa khoảng trắng thừa.
- LEFT / RIGHT / MID - Trích xuất text.
- LEN - Đếm số ký tự trong text.
- TODAY() - Trả về ngày hôm nay.
- ROUND(number, num_digits) - Làm tròn số.

Luyện tập
mỗi ngày!

★ Tham chiếu ô

Tham chiếu tương đối
(thay đổi khi copy) Ví dụ: A1

Tham chiếu tuyệt đối
(cố định khi copy) Ví dụ: \$A\$1

Tham chiếu hỗn hợp
(cố định một phần) Ví dụ: \$A1 hoặc A\$1

★ Sắp xếp & Lọc

- ✓ Sắp xếp dữ liệu A → Z hoặc Z → A
- ✓ Lọc để hiển thị dữ liệu cụ thể
- ✓ Dùng Custom Filter cho điều kiện nâng cao

★ Kiểu dữ liệu trong Excel

- Text - chữ hoặc ký tự
- Number - 10, 25, 3.14
- Date - 01-01-2024
- Boolean - TRUE / FALSE
- Blank - ô rỗng

★ Mẹo nhanh

- ✓ Dùng Ctrl + T để tạo Table
- ✓ Ctrl + Shift + L → Áp dụng / Bỏ Filter
- ✓ Ctrl + ; → Chèn ngày hiện tại
- ✓ Ctrl + Shift + \$ → Định dạng Currency
- ✓ Ctrl + Arrow → Nhảy đến dữ liệu cuối

CẨM NANG DATA ANALYST

TRANG 3

3. EXCEL CHO PHÂN TÍCH DỮ LIỆU

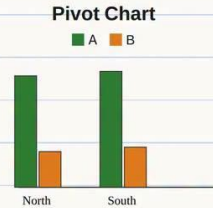
★ Pivot Table - Tóm tắt dữ liệu nhanh chóng

Pivot Table là gì?

Pivot Table là một tính năng mạnh mẽ của Excel dùng để tóm tắt, phân tích, khám phá và trình bày dữ liệu một cách có ý nghĩa.

Raw Data			
Date	Region	Product	Sales
01-01	North	A	1200
01-01	North	B	800
01-01	South	A	1500
01-02	North	A	1300
01-02	South	B	900
01-02	South	A	1100
...	...	...	...

Pivot Table			
Region	A	B	Grand Total
North	2500	800	3300
South	2600	900	3500
Grand Total	5100	1700	6800



Cách tạo Pivot Table?

1. Chọn vùng dữ liệu của bạn.
2. Vào Insert → PivotTable.
3. Chọn New Worksheet / Existing Worksheet.
4. Kéo các field vào Rows, Columns, Values và Filters.
5. Chọn giá trị tổng hợp như Sum, Count, Average, v.v.
6. Refresh dữ liệu khi cần.

Pivot Table Field List

☒ Date
☒ Region
☒ Product
☒ Sales
[More Tables...](#)

▽ Filters
≡ Rows
Region
T Columns
Product

Σ Values
Sum of Sales

Lợi ích chính

- ✓ Tiết kiệm thời gian
- ✓ Tóm tắt dễ dàng
- ✓ Báo cáo linh hoạt
- ✓ Insight tốt hơn

★ Pivot Chart - Trực quan hóa bản tóm tắt

■ A ■ B



Pivot Chart được tạo ra từ Pivot Table. Chúng giúp hiểu trực quan và trình bày dữ liệu hiệu quả. 🌟

★ Conditional Formatting

Highlight dữ liệu quan trọng bằng rules.

Ví dụ → Highlight top 10 giá trị hoặc giá trị > 1000

Sales	Status
1500	High
800	Low
1200	Medium
2000	High

★ Data Validation

Kiểm soát những gì người dùng có thể nhập vào một ô.

Ví dụ → Dropdown list

Product ▼
Laptop
Mobile
Tablet
Accessories

★ Remove Duplicates

Xóa các dòng trùng lặp để giữ dữ liệu sạch.
Data → Remove Duplicates → Select columns → OK

★ Shortcut hữu ích

Ctrl+T	→ Chuyển thành Table
Ctrl+Shift+L	→ Áp dụng/Bỏ Filter
Ctrl+Y	→ Redo
Ctrl+Z	→ Undo
Alt+=	→ AutoSum
Ctrl+Arrow	→ Nhảy đến vùng dữ liệu
Ctrl+Home	→ Về ô A1

★ Hàm Text bạn nên biết

LEFT(text, n) → Trích xuất ký tự bên trái
RIGHT(text, n) → Trích xuất ký tự bên phải
MID(text, start, n) → Trích xuất ký tự ở giữa
LEN(text) → Đếm số ký tự
TRIM(text) → Xóa khoảng trắng thừa

★ Hàm Date & Time

TODAY() → Ngày hôm nay
NOW() → Ngày & giờ hiện tại
DATE(2024,1,1) → Tạo ngày
YEAR(date) → Trích xuất năm
MONTH(date) → Trích xuất tháng

★ Hàm Logic

IF(condition, value_if_true, value_if_false)
AND(condition1, condition2, ...)
OR(condition1, condition2, ...)

CẨM NANG DATA ANALYST

TRANG 4

4. KIẾN THỨC NỀN TẢNG SQL

★SQL là gì?

SQL (Structured Query Language) được dùng để giao tiếp với các relational database. Nó giúp chúng ta lưu trữ, truy xuất, lọc và thao tác dữ liệu một cách hiệu quả.



1. SELECT

Dùng để truy xuất dữ liệu từ một table.

Example:

```
SELECT *  
FROM employees;
```

id	name	dept	salary
1	Asha	HR	50000
2	Ravi	IT	60000
3	Neha	Finance	55000

2. WHERE

Dùng để lọc dữ liệu dựa trên một điều kiện.

Example:

```
SELECT name, dept  
FROM employees  
WHERE dept = 'IT';
```

name	dept
Ravi	IT
Karan	IT

3. DISTINCT

Trả về các giá trị duy nhất (distinct).

Example:

```
SELECT DISTINCT dept  
FROM employees;
```

dept
HR
IT
Finance

4. ORDER BY

Dùng để sắp xếp kết quả.

Example:

```
SELECT name, salary  
FROM employees  
ORDER BY salary DESC;
```

name	salary
Neha	55000
Ravi	50000
Asha	45000

5. LIMIT

Giới hạn số dòng được trả về.

Example:

```
SELECT name, salary  
FROM employees  
LIMIT 2;
```

name	salary
Asha	45000
Ravi	50000

6. Column Aliases

Dùng để đặt tên tạm thời cho một column hoặc table.

Example:

```
SELECT name AS  
employee_name,  
salary AS  
monthly_salary  
FROM employees;
```

employee_name	monthly_salary
Asha	45000
Ravi	50000
Neha	55000

7. Toán tử so sánh

Toán tử	Ý nghĩa
=	Bằng
!= hoặc <>	Khác
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng

8. Toán tử Logic

Toán tử	Ý nghĩa
AND	Trả về true nếu cả hai điều kiện đều đúng
OR	Trả về true nếu có ít nhất một điều kiện đúng
NOT	Đảo ngược kết quả (true → false, false → true)

9. Kết hợp tất cả

Example:

```
SELECT name, dept, salary  
FROM employees  
WHERE salary > 50000  
AND dept != 'HR'  
ORDER BY salary DESC  
LIMIT 3;
```

10. Ví dụ thực tế

① Lấy tất cả employee

```
SELECT * FROM  
employees;
```

④ Lấy 5 employee có lương cao nhất

```
SELECT name, salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 5;
```

② Lấy tất cả employee thuộc phòng Finance

```
SELECT * FROM employees  
WHERE dept = 'Finance';
```

⑤ Đếm số lượng employee trong mỗi phòng ban

```
SELECT dept, COUNT(*)  
AS total_employees  
FROM employees GROUP BY  
dept;
```

★Điểm nhanh phòng vấn

- SQL không phân biệt hoa thường.
- Luôn kết thúc query bằng dấu chấm phẩy (;).
- Dùng tên alias có ý nghĩa.
- WHERE lọc dòng trước khi group.
- ORDER BY được thực thi sau cùng, ở cuối câu query.

CẨM NANG DATA ANALYST

TRANG 5

5. SQL AGGREGATION & GROUPING

★ Tại sao cần Aggregation?

Aggregation giúp chúng ta tóm tắt các tập dữ liệu lớn và trích xuất những insight có ý nghĩa.

Chúng ta dùng aggregate function cùng với GROUP BY để tóm tắt dữ liệu theo từng nhóm.

Sample Table: sales

id	name	dept	product	sales
1	Asha	HR	Laptop	50000
2	Ravi	IT	Laptop	60000
3	Neha	Finance	Mobile	30000
4	Karan	IT	Mobile	45000
5	Pooja	HR	Laptop	55000
...	...	...	...	...

Aggregate Functions

COUNT() → Đếm số dòng
SUM() → Tổng các giá trị
AVG() → Giá trị trung bình
MIN() → Giá trị nhỏ nhất
MAX() → Giá trị lớn nhất

1. Aggregate Functions

COUNT()

Đếm số lượng dòng.

Example:

```
SELECT COUNT(*)  
FROM sales;
```

SUM()

Tổng của column dạng số.

Example:

```
SELECT SUM(sales)  
FROM sales;
```

AVG()

Trung bình của column dạng số.

Example:

```
SELECT AVG(sales)  
FROM sales;
```

MIN()

Giá trị nhỏ nhất.

Example:

```
SELECT MIN(sales)  
FROM sales;
```

MAX()

Giá trị lớn nhất.

Example:

```
SELECT MAX(sales)  
FROM sales;
```

2. GROUP BY

Nhóm các dòng có cùng giá trị ở (các) column được chỉ định và áp dụng aggregate function trên từng nhóm.

Example:

```
SELECT dept,  
COUNT(*) AS  
total_emp,  
SUM(sales) AS  
total_sales  
FROM sales  
GROUP BY dept;
```

Result			
dept	total_emp	total_sales	
HR	2	105000	
IT	2	105000	
Finance	1	30000	

3. HAVING

Lọc các nhóm dựa trên điều kiện của aggregate.

Example:

```
SELECT dept, SUM(sales) AS total_sales  
FROM sales  
GROUP BY dept  
HAVING SUM(sales) > 80000;
```

4. WHERE vs HAVING

	WHERE	HAVING
Mục đích	Lọc dòng trước khi group.	Lọc nhóm sau khi group.
Dùng với	Dùng với mệnh đề WHERE.	Dùng với mệnh đề HAVING.
Aggregate	Không thể dùng aggregate function.	Có thể dùng aggregate function.

① Total sales per product

```
SELECT product, SUM(sales) AS total_sales  
FROM sales GROUP BY product;
```

② Average sale per department

```
SELECT dept, AVG(sales) AS avg_sales  
FROM sales GROUP BY dept;
```

5. Multiple Columns trong GROUP BY

Example:

```
SELECT dept, product, COUNT(*) AS total_orders,  
SUM(sales) AS total_sales  
FROM sales  
GROUP BY dept, product  
ORDER BY dept, total_sales DESC;
```

Result

dept	product	total_orders	total_sales
HR	Laptop	2	105000
HR	Mobile	0	0
IT	Laptop	1	60000
IT	Mobile	1	45000
Finance	Mobile	1	30000

★ SQL Interview - Bẫy & Mẹo

- ✓ Không dùng tên column trong SELECT nếu nó không nằm trong GROUP BY hoặc trong aggregate function.
- ✓ WHERE không thể dùng cùng với điều kiện aggregate.

7. Common Interview Queries

- Tìm phòng ban có tổng doanh số cao nhất.
- Tìm mức lương cao thứ hai trong mỗi phòng ban.

CẨM NANG DATA ANALYST

TRANG 6

6. SQL JOINS

★ Tại sao cần Joins?

Trong thực tế, dữ liệu được lưu trữ rải rác trên nhiều table. Joins giúp chúng ta kết hợp dữ liệu liên quan dựa trên một column chung giữa các table.

Primary Key & Foreign Key

Primary Key (PK) → Định danh duy nhất cho mỗi dòng trong một table.

Foreign Key (FK) → Column trong table này tham chiếu đến Primary Key của table khác.

employees			→		departments	
emp_id (PK)	name	dept_id (FK)			dept_id (PK)	dept_name
1	Asha	10			10	HR
2	Ravi	20			20	IT
3	Neha	10			30	Finance
4	Karan	30				

FK → PK

1. INNER JOIN



Chỉ trả về các bản ghi khớp nhau từ cả hai table.

Example:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Asha	HR
2	Ravi	IT
3	Neha	HR
4	Karan	Finance

2. LEFT JOIN



Trả về tất cả bản ghi từ table bên trái (A) và các bản ghi khớp từ table bên phải (B).

Example:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Asha	HR
2	Ravi	IT
3	Neha	HR
4	Karan	Finance

3. RIGHT JOIN



Trả về tất cả bản ghi từ table bên phải (B) và các bản ghi khớp từ table bên trái (A).

Example:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Asha	HR
2	Ravi	IT
3	Neha	HR
4	Karan	Finance

4. FULL OUTER JOIN



Trả về tất cả bản ghi từ cả hai table. Khớp nếu có thể, ngược lại là NULL.

Example:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Asha	HR
2	Ravi	IT
3	Neha	HR
4	Karan	Finance
NULL	NULL	Marketing

Nếu có phòng ban nào không có nhân viên, nó vẫn sẽ được trả về.

5. SELF JOIN

employees		
emp_id	name	manager_id
1	Asha	NULL
2	Ravi	1
3	Neha	1
4	Karan	2

Một table được join với chính nó.

Example:

```
SELECT e.emp_id, e.name,
       m.name AS manager_name
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

Result:

emp_id	name	manager_name
1	Asha	NULL
2	Ravi	Asha
3	Neha	Asha
4	Karan	Ravi

6. Join nhiều table

Example:

```
SELECT e.name, d.dept_name, s.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
LEFT JOIN salaries s ON e.emp_id = s.emp_id;
```

★ Câu hỏi phỏng vấn JOIN thường gặp

- Tìm nhân viên cùng tên phòng ban của họ.
- Tìm tất cả nhân viên, kể cả những người chưa được phân vào phòng ban nào.
- Tìm những phòng ban không có nhân viên nào.

CẨM NANG DATA ANALYST

TRANG 7

7. ADVANCED SQL BASICS

★ Tại sao cần Advanced SQL?

Giúp chúng ta giải quyết các vấn đề phức tạp, phân tích sâu hơn và viết query hiệu quả.

Sample Table: employees				
emp_id	name	dept_id	salary	hire_date
1	Asha	10	60000	2022-01-15
2	Ravi	20	55000	2021-06-10
3	Neha	10	70000	2023-02-20
4	Karan	30	50000	2020-12-01
5	Pooja	20	65000	2022-07-18
...	...	...	...	...

Key Takeaway

- Subqueries cho logic lồng nhau
- CASE cho logic điều kiện
- CTE để dễ đọc hơn
- Window Functions cho phân tích nâng cao

1. SUBQUERIES

Một query nằm bên trong query khác.

(a) Scalar Subquery

Trả về một giá trị duy nhất.

Example:

```
SELECT name, salary
FROM employees
WHERE salary >
(SELECT AVG(salary)
FROM employees);
```

(b) Multi-row Subquery

Trả về nhiều dòng. Dùng với IN, ANY, ALL.

Example:

```
SELECT name
FROM employees
WHERE dept_id IN
(SELECT dept_id
FROM employees
WHERE salary >
60000);
```

(c) Correlated Subquery

Subquery tham chiếu đến outer query, thực thi cho từng dòng.

Example:

```
SELECT e1.name,
e1.salary
FROM employees e1
WHERE e1.salary >
(SELECT
AVG(e2.salary)
FROM employees e2
WHERE e2.dept_id =
e1.dept_id);
```

2. CASE WHEN

Dùng để tạo điều kiện trong SELECT, WHERE, ORDER BY.

Example:

```
SELECT name, salary,
CASE
WHEN salary >= 60000 THEN 'High'
WHEN salary >= 50000 THEN 'Medium'
ELSE 'Low'
END AS salary_level
FROM employees;
```

3. COMMON TABLE EXPRESSION (CTE)

CTE (mệnh đề WITH) tạo ra một tập kết quả tạm thời có tên. Giúp query phức tạp dễ đọc hơn.

Example:

```
WITH high_earners AS (
SELECT name, salary
FROM employees
WHERE salary > 60000
)
SELECT * FROM high_earners
ORDER BY salary DESC;
```

CTE chỉ tồn tại trong một lần thực thi query. Dễ đọc hơn!

4. WINDOW FUNCTIONS (Concept)

Tính toán trên tập các dòng liên quan đến dòng hiện tại. KHÔNG gộp dòng như GROUP BY.

General Syntax:

```
<Function>() OVER (PARTITION BY column
ORDER BY column [ASC|DESC])
```

Example (Running Total):

```
SELECT name, salary,
SUM(salary) OVER (ORDER BY hire_date) AS
running_total
FROM employees;
```

5. ROW_NUMBER()

Gán số thứ tự duy nhất cho các dòng trong một partition.

Example:

```
SELECT name, dept_id, salary,
ROW_NUMBER() OVER
(PARTITION BY dept_id
ORDER BY salary DESC) AS rn
FROM employees;
```

Result (Example):

name	dept_id	salary	rn
Neha	10	70000	1
Asha	10	60000	2
Pooja	20	65000	1
Ravi	20	55000	2
Karan	30	50000	1

6. RANK()

Cho cùng rank với giá trị bằng nhau. Rank kế tiếp bị nhảy số.

Example:

```
SELECT name, dept_id, salary,
RANK() OVER (PARTITION BY dept_id
ORDER BY salary DESC) AS rnk
FROM employees;
```

Result (Example):

name	dept_id	salary	rnk
Neha	10	70000	1
Asha	10	60000	2
Pooja	20	65000	1
Ravi	20	55000	2
Karan	30	50000	1

7. DENSE_RANK()

Cho cùng rank với giá trị bằng nhau. Không có khoảng trống.

Example:

```
SELECT name, dept_id, salary,
DENSE_RANK() OVER (PARTITION BY
dept_id
ORDER BY salary DESC) AS drnk
FROM employees;
```

Result (Example):

name	dept_id	salary	drnk
Neha	10	70000	1
Asha	10	60000	2
Pooja	20	65000	1
Ravi	20	55000	2
Karan	30	50000	1

8. PARTITION BY vs GROUP BY

PARTITION BY (Window)	GROUP BY (Aggregate)
Giữ nguyên tất cả các dòng	Giảm số dòng (một dòng/nhóm)
Dùng với mệnh đề OVER	Dùng với aggregate function

9. Ví dụ thực tế

① Top 3 nhân viên lương cao nhất mỗi phòng ban

```
SELECT name, dept_id, salary
FROM (SELECT *, ROW_NUMBER() OVER
(PARTITION BY dept_id ORDER BY
salary DESC) AS rn
FROM employees ) t
WHERE rn <= 3;
```

★ Câu hỏi phỏng vấn Advanced SQL thường gặp

- Tìm mức lương cao thứ hai trong mỗi phòng ban.
- Tìm top N mức lương trong mỗi phòng ban.
- Tổng lũy kế (running total) của salary theo hire_date.

CẨM NANG DATA ANALYST

TRANG 8

8. STATISTICS CHO DATA ANALYST

★ Tại sao Statistics quan trọng?

Statistics giúp chúng ta hiểu được dữ liệu, nhận biết quy luật, đo lường độ biến thiên và đưa ra quyết định dựa trên dữ liệu tốt hơn.

Population vs Sample

Population: Toàn bộ tập hợp các phần tử mà chúng ta muốn nghiên cứu.

Sample: Tập con của population được dùng để rút ra insight.

Key Terms

- **Parameter** → giá trị từ population
- **Statistic** → giá trị từ sample
- **Descriptive Stats** → tóm tắt dữ liệu
- **Inferential Stats** → khái quát hóa từ sample sang population

1. MEAN (Trung bình)

Tổng tất cả giá trị chia cho số lượng.

Example:

Data: 10, 20, 30, 40, 50

Mean = $(10+20+30+40+50)/5 = 150/5 = 30$

2. MEDIAN (Giá trị giữa)

Giá trị ở giữa khi dữ liệu đã sắp xếp.

• Số lượng lẻ → số ở giữa

• Số lượng chẵn → TB hai số giữa

Example:

Odd: 5, 7, 8, 9, 12 → Median = 8

Even: 2, 4, 6, 8, 10, 12 → $(6+8)/2 = 7$

3. MODE (Giá trị xuất hiện nhiều nhất)

Giá trị xuất hiện nhiều nhất.

Example:

Data: 2, 3, 3, 3, 5, 7, 8 → Mode = 3

Dữ liệu có thể có không, một hoặc nhiều mode. ☹

4. RANGE

Hiệu số giữa giá trị lớn nhất và nhỏ nhất.

Range = Max - Min

Example:

Data: 3, 7, 9, 12, 18 → Range = $18 - 3 = 15$

5. VARIANCE (σ^2)

Đo mức độ phân tán của các điểm dữ liệu so với mean.

Example:

Data: 2, 4, 4, 5, 5, 7, 9

Mean = 5, Variance (σ^2) = 4

6. STANDARD DEVIATION (σ)

Căn bậc hai của variance.

Example:

Từ variance ở trên = 4

Standard Deviation (σ) = $\sqrt{4} = 2$

σ thấp → dữ liệu gần mean. σ cao → phân tán nhiều hơn.

7. PERCENTILES & QUANTILES

• Percentile: giá trị mà dưới nó có % quan sát nhất định.

• Quartiles chia dữ liệu thành 4 phần bằng nhau.

Đo lường	Ý nghĩa
Q1 (25th Pctl)	25% dữ liệu nằm dưới Q1
Q2 (50th Pctl)	Giống Median
Q3 (75th Pctl)	75% dữ liệu nằm dưới Q3

Data (sorted): 10, 20, 30, 40, 50, 60, 70, 80

Q1=22.5, Q2(Median)=45, Q3=67.5

8. IQR & OUTLIERS

IQR = Q3 - Q1

Outliers: giá trị nằm xa các điểm dữ liệu khác.

Rule (Tukey's Rule):

Lower = Q1 - $1.5 \times \text{IQR}$

Upper = Q3 + $1.5 \times \text{IQR}$

Q1=22.5, Q3=67.5 → IQR=45

Lower=-45, Upper=135 → outlier nếu <-45 hoặc >135

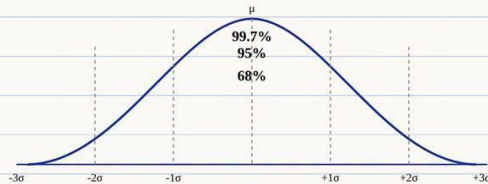
9. NORMAL DISTRIBUTION

• Dữ liệu theo đường cong hình chuông đối xứng.

• Mean = Median = Mode.

• ~68% dữ liệu trong 1 σ quanh mean.

• ~95% trong 2 σ . ~99.7% trong 3 σ .



10. SKEWNESS (Hình dạng phân phối)

(a) Symmetric

(Skewness = 0)

(b) Right Skewed

(Skewness > 0)

(c) Left Skewed

(Skewness < 0)

VÍ DỤ THỰC TẾ TRONG KINH DOANH

Mean Sales → Hiệu suất trung bình

Std. Deviation của Delivery Time → Tính ổn định

Outliers trong Transactions → Phát hiện gian lận

★ ĐIỂM NHANH PHÒNG VẤN STATISTICS

- Luôn hiểu rõ ý nghĩa thực sự của metric.
- Nhớ vững các công thức chính.

CẨM NANG DATA ANALYST

TRANG 9

9. PYTHON CHO DATA ANALYSIS

Tại sao dùng Python cho Data Analysis?

- Dễ học và rất mạnh mẽ.
- Hệ sinh thái thư viện dữ liệu khổng lồ.
- Tuyệt vời cho làm sạch, phân tích, trực quan hóa dữ liệu và ML.
- Mã nguồn mở và được dùng rộng rãi trong ngành.

Python Data Stack



Python + Thư viện = Phân tích mạnh mẽ ☺

Quy trình điển hình

1. Import thư viện
2. Load dữ liệu
3. Khám phá & làm sạch
4. Phân tích / Biến đổi
5. Trực quan hóa
6. Rút ra Insight

1. PYTHON BASICS (dành cho Analyst)

- **Variables** → name = value
- **Data Types** → int, float, str, list, dict, bool
- **Operators** → +, -, *, /, //, %, **
- **Comparison** → ==, !=, >, <, >=, <=
- **Logical** → and, or, not
- **Functions** → def my_func(): pass
- **Loops** → for, while
- **List Comprehension** → [x for x in l if x>0]

Example:

```
>>> a = 10
>>> b = 3
>>> a + b
13

>>> a // b
3

>>> [x*x for x in
range(5)]
[0, 1, 4, 9, 16]
```

2. NUMPY BASICS

NumPy → tính toán số học trong Python. Hỗ trợ array đa chiều, kích thước lớn. Các hàm toán học mạnh mẽ.

```
import numpy as np
arr = np.array([1,2,3,4,5])
arr2 = np.array([[1,2,3],
[4,5,6]])
arr.shape # (5,)
arr2.shape # (2, 3)
arr2.mean() # 3.0
```

3. PANDAS INTRODUCTION

pandas → thư viện thao tác và phân tích dữ liệu. Hai cấu trúc dữ liệu chính:

1. **Series** → array 1 chiều có label
2. **DataFrame** → bảng 2 chiều có label (giống table SQL)

```
s = pd.Series([10,20,30], index=['a','b','c'])
```

4. ĐỌC DỮ LIỆU

pandas có thể đọc nhiều định dạng file. Phổ biến: CSV, Excel, JSON, SQL, v.v.

```
df_csv = pd.read_csv('data.csv')
df_excel = pd.read_excel('data.xlsx')
df_head = df_csv.head() # 5 dòng đầu
df_info = df_csv.info() # thông tin tổng quan
df_shape = df_csv.shape # (dòng, cột)
```

5. CHỌN & LỌC DỮ LIỆU

```
df['Name'] # một column
df[['Name','Salary']] #
nhiều column
df[df['Salary'] > 55000]
# lọc dòng
```

6. loc vs iloc

loc → dựa trên label
iloc → dựa trên vị trí

```
df.loc[0] # dòng đầu
df.iloc[0,1] # dòng 0,
cột 1
```

7. GROUPBY()

Dùng để split-apply-combine.

```
df.groupby('Dept')
['Salary'].mean()
```

8. MERGE() / JOIN

Kết hợp các DataFrame.

```
pd.merge(left_df,
right_df,
on='id', how='inner')
```

9. CONCAT()

Nối các DataFrame.

```
pd.concat([df1, df2])
```

10. KIỂM TRA DỮ LIỆU CƠ BẢN

```
df.head() # 5 dòng đầu
df.tail() # 5 dòng cuối
df.info() # thông tin tổng
quan
```

★ Câu hỏi phỏng vấn pandas thường gặp

- Bạn xử lý missing values như thế nào?
- Sự khác biệt giữa loc và iloc là gì?

10. DATA CLEANING + EXPLORATORY DATA ANALYSIS (EDA)

★ Data Cleaning là gì?

Data cleaning là quá trình phát hiện và sửa (hoặc xóa) dữ liệu sai, thiếu, không liên quan, trùng lặp hoặc sai định dạng nhằm cải thiện chất lượng và độ tin cậy.

Data Quality Dimensions

- Accuracy**
Dữ liệu chính xác và đáng tin cậy.
- Completeness**
Tất cả dữ liệu cần thiết đều đầy đủ.
- Consistency**
Không có mâu thuẫn trong dữ liệu.
- Timeliness**
Dữ liệu được cập nhật mới nhất.
- Validity**
Dữ liệu tuân theo các rule & ràng buộc.

EDA Workflow

1. Xác định mục tiêu
2. Import dữ liệu
3. Hiểu dữ liệu
4. Làm sạch dữ liệu
5. Khám phá dữ liệu
6. Trực quan hóa Insight
7. Tóm tắt kết quả

1. Missing Values

Identify → Handle → Validate

Identify:

```
df.isnull().sum()
```

Handle:

- Xóa dòng / column
- Điền bằng mean/median
- Forward / Backward fill
- Điền bằng giá trị cố định

2. Duplicate Records

Phát hiện và xóa các dòng trùng lặp.

Identify:

```
df.duplicated().sum()
```

Remove:

```
df.drop_duplicates()
```

3. Dữ liệu sai / không nhất quán

Sửa lỗi chính tả, cách viết không nhất quán, giá trị sai.

City
delhi
DELHI
Dehli

City
Delhi
Delhi
Delhi

4. Chuyển đổi kiểu dữ liệu

Chuyển sang đúng kiểu dữ liệu.

```
df['Age'] = df['Age'].astype(int)
```

5. Phát hiện & xử lý Outlier

Phát hiện giá trị bất thường lệch khỏi quy luật thống thường.

Methods:

- IQR Method
- Z-Score Method
- Boxplot
- Domain Rules

8. EDA - Các loại phân tích

A. Univariate Analysis

Nghiên cứu từng biến một.

Ví dụ (Salary): Summary Statistics, Histogram, Boxplot

B. Bivariate Analysis

Nghiên cứu mối quan hệ giữa hai biến.

Ví dụ: Salary vs Experience - Scatter Plot



C. Multivariate Analysis

Nghiên cứu mối quan hệ giữa 3 biến trở lên.

Ví dụ: Salary, Experience, Age - Pairplot

Correlation Basics

Range: -1 đến +1
+1 → Tương quan dương mạnh
0 → Không có quan hệ tuyến tính
-1 → Tương quan âm mạnh
Dùng heatmap để trực quan hóa correlation.

9. Case Study nhỏ (Đơn hàng E-Commerce)

OrderID	Customer	City	Product	Qty	Price	OrderDate	Age
1001	Asha	Delhi	Phone	1	15000	2024-01-05	25
1002	Ravi	delhi	Laptop	1	55000	2024-01-06	NaN
1003	Neha	Mumbai	Phone	2	15000	2024-01-06	30
1002	Ravi	Delhi	Laptop	1	55000	2024-01-06	NaN
1004	Karan	Bengaluru	Headphone	1	2000	2024-01-07	28
1005	Pooja	Chennai	Phone	1	15000	Invalid Date	22

Vấn đề phát hiện

- Missing Age
- Dòng trùng lặp
- Không nhất quán City
- Sai định dạng ngày

Các bước làm sạch

1. Điền Age thiếu bằng median
2. Xóa dòng trùng
3. Chuẩn hóa City
4. Sửa OrderDate

Kết quả

→ Dữ liệu chất lượng cao, sẵn sàng cho EDA

★ CHECKLIST PHÒNG VẤN PHẦN 1 (CUỐI)

- ✓ Hiểu sâu kiến thức nền tảng SQL
- ✓ Viết được query phức tạp (JOIN, GROUP BY, HAVING)
- ✓ Thành thạo subqueries & window functions
- ✓ Nắm vững statistics & probability cơ bản
- ✓ Vững Python + Pandas để xử lý dữ liệu
- ✓ Trước khi làm EDA, đảm bảo dữ liệu sạch & đáng tin cậy